



House Rules — Complete User Guide

A beginner-friendly guide: AI manages your company's "house rules" For erpilot v3.25+ Reading time: 15 min



What are "House Rules"? Why do you need them?

Just like every household has rules (no snacks after 11pm, wash hands at home), **every company has its own "business rules"**:

- PO over NT\$100k → boss approves
- WO without Recipe → cannot release to factory floor
- SO discount over 5% → manager reviews
- Customer credit limit exceeded → no new order
- Invoice must be issued before shipping

Traditional approach: **rules live in employees' heads**. Problems:

- 😬 New hires don't know
- 😬 Veterans forget
- 😬 No handoff between roles
- 😬 People sneak around when boss is away
- 😬 Hard to find accountability after incidents

erpilot's "House Rules" **put rules into the system**:

- ✅ Employees follow rules — error rate drops 80%
- ✅ Boss sleeps soundly (system auto-blocks violations)
- ✅ No retraining when staff changes (rules live in system)
- ✅ Manager can emergency-override with audit log
- ✅ ISO / GMP audits → one-click full trail export

vs How competitors do it (and why erpilot is different)

ERP	How to set rules	Drawback
SAP B1	Consultant changes code	Wait 1 month + cost NT\$50-200k
鼎新	Hidden config screens	Rigid conditions; can't handle edge cases
NetSuite	Write JavaScript (SuiteScript)	Beginners can't do it
Odoo	Python expressions	Dangerous + requires coding
erpilot ✨	👉 Choose any of 3	Anyone can edit, instant effect

🚀 3 Ways to Set House Rules in erpilot

Method A: Click in the UI (easiest)

1. Log in
2. Sidebar → "✅ Approvals" (House Rules share this page in v3.25)
3. Switch to "⚙️ Rules" tab
4. Click "➕ Add Rule"
5. Fill the form:

```

Name:      No PO over $50k
Trigger:   [PO Create ▼]
Condition: [field_compare ▼]
  Field:    amount
  Operator: <= (less or equal)
  Value:    50000
Action:    [block ▼]
Message:   Amount too high, please split
Override:  [manager ▼]
Active:    ☒
          [✓ Save] [X Cancel]

```

6. Save → Instantly effective (no restart, no deploy)

Method B: Talk to AI (natural language — erpilot's signature)

Don't want to fill forms? Just talk:

1. Click "🗨️ AI Assistant" on sidebar
2. Type:

"Our company should require manager approval when SO discount exceeds 5%"

3. AI responds with ConfirmCard:

```
🗨️ I'll set this house rule:  
  
Name:      SO discount > 5% needs mgr  
Trigger:   Sales Order create  
Condition: discount_pct > 0.05  
Action:    require_approval  
Override:  manager  
  
[✓ Confirm] [X Cancel]
```

4. Click [✓ Confirm] → Rule goes live
5. Next time a salesperson creates an SO with >5% discount → auto-routed to approval

🚀 This is erpilot's most powerful feature: describe needs in plain words, AI translates to system rules. No SQL, no JavaScript, no Python required.

Method C: API / Plugin (for engineers)

Need a custom condition (e.g., credit limit algorithm)? Engineers can plug in new condition types:

```
from app.services.policy_engine import register_condition  
  
async def check_credit_limit(params, context, db):  
    """Check customer credit limit (plugin example)"""  
    from app.models.crm_sales import Customer  
    from sqlalchemy import select  
    cust_id = context.get('customer_id')  
    if not cust_id:  
        return False  
    cust = (await db.execute(  
        select(Customer).where(Customer.id == cust_id)  
    )).scalar_one_or_none()  
    if not cust:  
        return False  
    return context.get('amount', 0) <= cust.credit_limit  
  
register_condition("credit_check", check_credit_limit)
```

After this, users will see "credit_check" as a condition option in the UI.

The 4 Building Blocks of a House Rule

1 Trigger

"What action triggers this rule?" 16+ built-in triggers:

Category	Triggers	When
Production	<code>wo.release</code> / <code>wo.complete</code> / <code>wo.cancel</code>	WO actions
Purchase	<code>po.create</code> / <code>po.approve</code> / <code>po.receive</code> / <code>po.cancel</code>	PO actions
Sales	<code>so.create</code> / <code>so.confirm</code> / <code>so.ship</code> / <code>so.cancel</code>	SO actions
Inventory	<code>inventory.delete</code> / <code>inventory.transfer</code>	Delete part / transfer
Accounting	<code>journal.post</code> / <code>ar.create</code> / <code>ar.collect</code>	Posting / invoicing / collection
CRM	<code>lead.convert</code> / <code>opportunity.stage_changed</code>	Sprout convert / Chase advance

2 Condition

"When does the rule apply?" 5 built-in types:

Type	Use	Example
always	Always trigger	"PO must always have remark"
has_bom	Pass only if Recipe exists	"WO release needs Recipe"
field_compare	Compare fields	"amount > 100k"
count_check	Count list length	"PO at least 1 item"
custom	Plugin-defined	"Credit limit check"

`field_compare` operators: `gt` / `gte` / `lt` / `lte` / `eq` / `ne`

3 Action

"What happens on violation?" 4 types:

Action	Behavior	Use case
 allow	Just pass (log only)	Stats tracking
 warn	Don't block; show UI toast	"Amount large, please confirm"
 block	Must meet condition to continue; can override	"Recipe required"
 require_approval	Enter approval workflow	"PO > 100k"

4 Override Role

"Who can release blocked actions?" Common values:

- `manager`
- `admin`
- `null` — no one can override (strictest)

When blocked, UI shows "🔒 Manager Override" button. The role-holder must click + provide reason to release.



Default House Rules (installed automatically)

Out-of-the-box, erpilot ships with 3 common rules:

```
● WO release requires "Recipe"
trigger: wo.release
condition: has_bom
action: block
override: manager
message: "Product has no Recipe yet. Please go to Production → Edit Recipe. Or ask manager to
override."

● PO > NT$100k needs manager approval
trigger: po.create
condition: amount > 100000
action: require_approval
override: manager

● PO must have at least 1 item
trigger: po.create
condition: items >= 1
action: block
override: none
```

Don't want these? Just toggle `is_active=false` in UI. No code change, no deploy, no consultant.

Real Scenario: Buyer's Day

Scenario: Create a high-value PO

1. 9:00 Buyer opens erpilot "Purchase" page
2. Clicks " + Quick Create"
3. Fills: Supplier = ChangJiang / Part = M6 Bolt / Qty = 1000 / Price = 150
4. Saves → System calculates total = NT\$150,000
5. 9:01 System auto-runs policy evaluate:
 - "PO > 100k needs manager approval" hits (150,000 > 100,000)
 - Action = require_approval
6. Buyer sees:

"  PO-2026-0042 saved as draft, pending manager approval"
7. 9:05 Plant Manager receives notification
8. Manager opens "  Approvals" → sees this PO in "pending my approval"
9. Clicks "✓ Approve" → adds comment "OK, this supplier has good credit" → confirms
10. PO becomes approved; buyer can proceed with receiving flow

Throughout the process:

-  Zero hardcoded
-  Zero consultant fee
-  Zero training
-  Complete audit log (who approved / when / comment)

Scenario: Emergency override

One day, a customer urgently needs goods, but the product has no Recipe yet:

1. Manager tries to release WO → Blocked
2. System message: "Product has no Recipe. **Or ask manager to override.**"
3. Manager clicks " Manager Override"
4. Input box: "Please enter override reason (will be audited)"
5. Manager fills: "Customer urgent, release now, will add Recipe tomorrow"
6. Confirm → WO released + audit log records "Manager X overrode rule Y, reason: customer urgent"

Full compliance, sufficient flexibility.

Audit Log (for compliance / debugging)

Every policy evaluation writes to `PolicyAuditLog` :

- Which rule was triggered
- Result (`allowed` / `blocked` / `overridden` / `warned`)
- Context (PO amount / customer ID — **sensitive data auto-truncated**)
- Who triggered / when
- Override user + reason (if any)

For ISO 9001 / GMP / FDA / food safety compliance. Query: `GET /api/policies/audit?rule_id=xxx&trigger=wo.release`

FAQ

Q1: Can different tenants (companies) have different rules?

- ✓ Yes. `PolicyRule` includes `tenant_id` ; multi-tenant isolation is natural.

Q2: How are conflicting rules resolved?

For the same trigger, rules are evaluated by **priority ascending**. First **block** wins. Convention: **priority < 50** for system rules, **50-200** daily, **200+** special cases.

Q3: Can I write complex Python conditions?

Don't directly **eval** Python strings (unsafe). Use **register_condition()** plugin instead: write a Python function + name + register → it appears in UI.

Q4: Can I withdraw a rule?

✓ Two ways:

- Set **is_active=false** (recommended; preserves history)
- DELETE removes it (audit log still preserved)

Q5: Won't managers abuse override?

Every override writes audit log; review periodically. You can add stricter rules:

- E.g., "must email boss before overriding"
- E.g., "max 5 overrides per month per manager"

Q6: Will rules block system background tasks?

No. erpilot only runs House Rules on **user-initiated** actions. System cron / cleanup / migration don't trigger.

Q7: I can't read English trigger names (po.create / so.confirm)...

i18n translations coming. Current list has Chinese descriptions. When using AI to author rules, **just talk in plain language**; AI translates trigger names.

Q8: Can I export/import rules (company move / multi-plant deploy)?

Yes via **GET /api/policies/rules** returning JSON; **POST** to load into a new env. Full import/export UI is Phase 2.

🔧 For Engineers: Technical Details

Schema

```
class PolicyRule(Base, TenantMixin):
    id: str (UUID PK)
    name: str (max 200)
    description: text
    trigger: str (whitelisted, 16+)
    condition_type: str (5 built-in + plugins)
    condition_params: JSON
    action: str (allow/warn/block/require_approval)
    message: str
    override_role: str | None
    is_active: bool (default True)
    priority: int (default 100)
    created_by / created_at / updated_at

class PolicyAuditLog(Base, TenantMixin):
    id, rule_id, trigger, action_taken
    context: JSON (auto-truncated)
    user_id, override_by, override_reason
    created_at
```

Integration in service layer

```
# Before (v3.24, hardcoded)
if not bom:
    raise BusinessRuleError("Need BOM")

# After (v3.25, data-driven)
result = await evaluate_policies(db, "wo.release", {"product_id": ...})
if result.blocked:
    raise BusinessRuleError(
        result.message,
        can_override=result.can_override,
        override_role=result.override_role,
    )
```

Plugin new condition

```
from app.services.policy_engine import register_condition

async def my_custom(params: dict, context: dict, db: AsyncSession) -> bool:
    # Return True = condition holds, rule passes
    # Return False = condition fails, action triggers
    return ...

register_condition("my_custom", my_custom)
```

Why this is strategically important for erpilot

"Hardcoded rule = customer must edit code = it's not SaaS"

Previously erpilot hardcoded "WO release needs BOM" — as rigid as SAP/鼎新. From v3.25 onward, with PolicyEngine:

-  Users toggle rules in UI → 0 deploy / 0 consultant fee
-  AI authors rules → even beginners can customize
-  Plugin mechanism → special countries / industries extensible
-  Audit log → ISO/GMP/FDA compliant
-  Manager override → flexibility + paper trail

This is erpilot's real competitive advantage vs SAP/鼎新 — something they can't do, we do: end users customize their own business rules.

Related Docs

- [USER_MANUAL_EN.md](#) — Full user manual
- [HOW_TO_GET_LLM_API_KEY_EN.md](#) — Enable AI chat
- Chinese version: [HOUSE_RULES_GUIDE_ZH.md](#)

v3.25 (2026-05-18) · erpilot original design · World's first AI-conversational ERP rule engine